Algorithm Logic

Problem: when seeing a word like "_ a _ _ e" one might guess TABLE, MAPLE, CABLE, etc. The smartest approach would be to guess the most frequent letter within that "dictionary" of possible words. This kind of sums up our proposed algorithm.

Preliminary Idea:

- At any point in the game, the state of the hidden word will be "_ _ a _ _ e"
- We want to keep track of the wrong letters that we've guessed, like: { t, s, r }
- Since we're given a dictionary of words that the hidden words may be from, we can separate the dictionary into candidates and non-candidates. Then, we'll only guess the most frequent letter that's present in the candidate sub-dictionary.
- We can categorize a word as a candidate if and only if:
    1.) It's the right length
    2.) It matches the revealed spots
    3.) It doesn't contain any letters from our wrong-guess list
    4.) It doesn't have a revealed letter in a not-yet-revealed spot
- So our basic algorithm will be something like:
    - Preprocess (need to figure out HOW)
    - keep a list of candidate words
    - on each turn:
        - for every letter we haven't guessed yet:
            - count how many candidates contain that letter (letter frequency)
        - guess the most frequent letter * not total letter count since duplicate letters don't matter (next page)
        - update candidate list based on result of guess

Algorithm Logic cont.

*

- If we have a candidate list that looks like: {banana, apple, grape, grain} we don't want to count the most frequent letter by total occurrences since guessing a letter reveals it everywhere.
- By that logic the frequencies are:
  - A: appears in 3 words
  - E: appears in 2 words
  - N: appears in 2 words ... etc.
- After guessing A, we can use a "most frequent English letters list" or something to tiebreak our next guess.

Data Structures for Project

1) Preprocess word list by length w/ map (key, val) as map(word length, list of words of that length) * might take a LOT of space

2.) Preprocess word list into binary values since String objects take ~2 bytes per character and an int is always 4 bytes (after counting length and whatever)

3.) Perhaps binary conversion AFTER preprocessing into length-based list